

Antal International Java Backend Developer - What to Learn

Overview

This is a heavily Java/Spring/enterprise-tooling JD — a bigger stretch from your React/Node/TypeScript background than most JDs you've tailored for. Your CS fundamentals (Java from coursework, strong DSA, SQL, REST API design from Node work) transfer conceptually, but the specific frameworks and enterprise tools below are largely new. Prioritize MUST items — they're the core of the role.

MUST learn

Spring Boot (and the Spring ecosystem: Spring Cloud, Spring Security)

- What it is: Spring Boot is the dominant Java framework for building REST APIs and backend services — auto-configuration, embedded servers, dependency injection via annotations (`@RestController`, `@Service`, `@Autowired`). Spring Cloud adds microservices patterns (service discovery, config server, circuit breakers); Spring Security handles auth/authz.
- Why the JD wants it: it's the #1 listed skill — this is fundamentally a Spring Boot role.
- The 20%: build one small REST API in Spring Boot (a CRUD app with a controller, service, and repository layer talking to Postgres via Spring Data JPA). This single exercise covers most interview-level questions.
- Bridge: the controller/service/repository layering maps directly onto your Express.js route/controller/model pattern; `@Autowired` dependency injection is the annotation-driven version of importing/instantiating a service in Node.

Streaming & messaging: Kafka (primary), Apache Flink/Storm (secondary)

- What it is: Kafka is a distributed event-streaming platform — producers publish messages to topics, consumers read them, enabling async, decoupled communication between services. Flink/Storm are stream-processing engines that consume from Kafka to transform data in real time.
- Why the JD wants it: listed explicitly as a required streaming framework — backend/payments systems often use Kafka for event-driven architectures (e.g. order events, audit logs).
- The 20%: understand producer/consumer/topic/partition/consumer-group vocabulary and why Kafka is used over a plain REST call (decoupling, replay, backpressure). Flink/Storm can stay conceptual — know they process Kafka streams; don't need hands-on depth for a fresher role.
- Bridge: closest analog in your own work is Inngest (Vibe project) — durable, async background jobs. Kafka is the same "decouple work from the request" idea, generalized to inter-service messaging instead of one app's background jobs.

Enterprise DevOps/observability tooling: Jenkins, SonarQube, Splunk/Kibana/Grafana/Prometheus, GitLab

- What it is: Jenkins = CI/CD pipeline automation (build/test/deploy on every commit). SonarQube = static code-quality/security scanning. Splunk/Kibana/Grafana/Prometheus = log aggregation and metrics dashboards for monitoring production systems.
- Why the JD wants it: enterprise teams standardize on this tooling for release + observability at scale — it's process/tooling knowledge, not deep engineering.
- The 20%: know what each tool's *category* is (CI/CD, code quality, logs vs. metrics) and roughly how they fit a pipeline: commit -> Jenkins build/test -> SonarQube quality gate -> deploy -> Prometheus/Grafana monitor -> Splunk/Kibana log search. You don't need hands-on — name-recognition + workflow fluency is enough for a fresher screen.
- Bridge: you already use Sentry (error tracking) and Mixpanel (analytics dashboards) at Clink AI — same idea of "observe the system in production," just enterprise-flavored tools.

Security scanning & standards: OWASP, Fortify, Veracode

- What it is: OWASP Top 10 is the standard list of common web vulnerabilities (SQL injection, XSS, broken auth, etc.). Fortify/Veracode are automated static/dynamic security-scanning tools that check code against these.
- Why the JD wants it: payments-domain-adjacent backend work has strict security compliance requirements.
- The 20%: skim the OWASP Top 10 list once — you'll recognize most of it from general secure-coding practice (input validation, parameterized queries, auth checks) which you already apply informally. Fortify/Veracode are tool names to recognize, not to learn deeply.

GOOD to learn

API tooling: Postman, Swagger/OpenAPI, Apigee, Castlemock

- What it is: Postman (API testing — you likely already use this informally), Swagger/OpenAPI (machine-readable API spec + auto-generated docs), Apigee (Google's API gateway/management platform), Castlemock (API mocking).
- Why the JD wants it: "evaluate third-party APIs for integration" + general API-lifecycle tooling.
- The 20%: if you haven't used Postman, spend 15 minutes with it. Swagger/OpenAPI is worth a quick look since it's common even outside enterprise Java (annotate a REST endpoint, get an auto-generated spec/docs page).

NoSQL & caching depth: Memcache (alongside Redis, which you know)

- What it is: Memcache is an older, simpler in-memory key-value cache (no persistence, no data structures) — Redis is the modern superset you already know from Echo/Vibe.
- Why the JD wants it: legacy enterprise systems often still run Memcache alongside newer Redis deployments.
- The 20%: just know Memcache is "Redis without persistence or rich data types" — your Redis knowledge already covers the concept.

Not worth deep prep

Oracle (vs. PostgreSQL/MySQL you know — same relational SQL concepts, different vendor syntax); Protobuf (binary serialization alternative to JSON — know it exists, skip hands-on); payments-domain knowledge (mentioned as "preferred," not required — don't chase domain expertise for a fresher screen).